

Online Exam Integrity Monitoring System

Project and Seminary
BSc in Computer and Software Engineering

Dinis Contreiras
A50496@alunos.isel.pt
934671411

Miguel Sanches
A49337@alunos.isel.pt
919772040

Supervisor:
Prof. Jorge Martins
jorge.martins@isel.pt

March 2026

1 Context

The widespread adoption of digital platforms in academic settings has transformed the way exams are performed, offering significant advantages when it comes to automation, accessibility, and even administrative efficiency for both educators and students. This transition comes with associated costs and new challenges, particularly the preservation of academic integrity.[1] When exams move from the paper world to laptops and computers, even with educators physically present in the room, the screen essentially becomes a private space for the student. This creates a meaningful gap where the professor is present but effectively blind to what is happening on each student's machine. Our solution aims to fill this gap by providing educators with a real-time window into each student's machine, extending visibility without requiring complex infrastructure or imposing an excessive burden on students.

2 Solution

Our solution consists of two main components: a lightweight daemon service that runs silently on the student's machine during execution of the exam and a professor console connected through a central server. The daemon monitors and actively mitigates suspicious behavior, detecting focus loss, forbidden processes, clipboard usage, and network changes, while a heartbeat mechanism ensures the professor is alerted in case a client stops running.[2] The exam itself is served through a web application where students authenticate with their institutional email and professors manage the full exam life cycle.

3 System Requirements

3.1 Functional Requirements

- Professors can create and manage exams, start sessions and generate a session code for students to join.
- Students authenticate with their institutional email and access the exam through a web application.
- The daemon detects suspicious behavior including window focus loss, forbidden processes, clipboard usage and network changes.
- The daemon actively blocks clipboard access and attempts to terminate forbidden processes.
- A heartbeat mechanism detects client crashes or forced termination.
- Events are relayed in real time to the professor console, displaying each student's status and a timeline of occurrences
- Session events are persistently logged on the server for later review

3.2 Non-Functional Requirements

- Session data and events transmitted between components should be secure, student authentication should be properly handled.
- The server should support multiple simultaneous exam sessions with a large number of concurrent students.
- The system should handle brief network interruptions gracefully without losing events.
- The project should be well documented to ensure maintainability and extensibility.

3.3 Optional Features

- Force full screen and minimize block during the exam.
- Block window switching.
- Block system shortcuts.

4 Architecture

The system is made of three main components. On each student machine, a background daemon written in C# responsible for monitoring and mitigating suspicious behavior during an exam. Alongside the daemon, the student can access the exam through a web browser. On the professor's side, a web browser provides access to the professor console, where the status of each student is displayed in real time. Both the student daemon and professor console communicate with a central server build with Ktor in Kotlin, responsible for relaying events via WebSockets[3], exposing a REST API for authentication and exam management, and persisting data in a SQLite database.[4] The server architecture ensures no single participant's machine bears the responsibility of keeping the session running and that all events are delivered reliably.

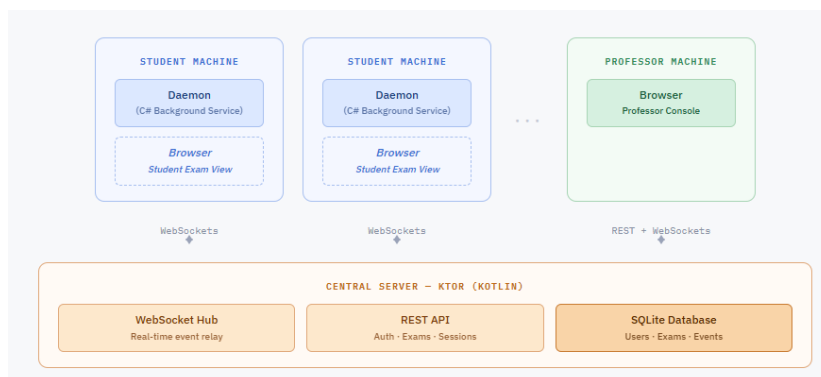


Figure 1: System Architecture

5 Challenges

The primary risk of this project lies in the adoption of C# and the .NET ecosystem which the team is not familiar with. Beyond the language itself, the core monitoring features of the daemon require calls to the native Windows APIs through P/Invoke.[5] This is technically demanding work that the team has no prior experience with, and may introduce unexpected additional complexity during development.

6 Plan

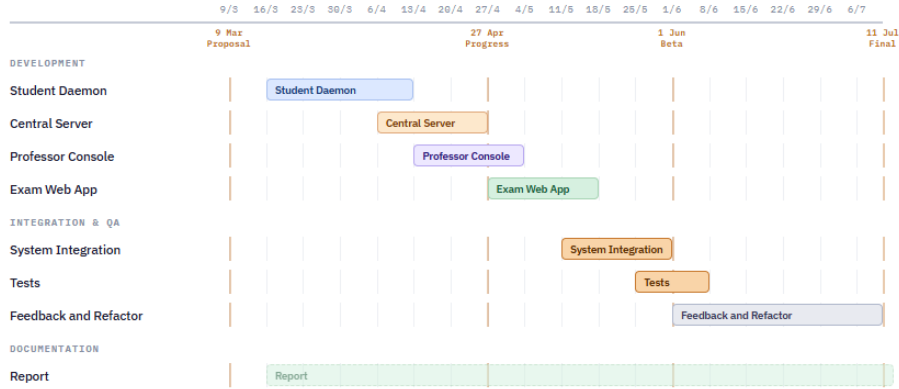


Figure 2: Project Plan

7 References

- [1] C. Mutimukwe, O. Viberg, C. McGrath, and T. Cerratto-Pargman, “Privacy in online proctoring systems in higher education: stakeholders’ perceptions, awareness and responsibility,” *Journal of Computing in Higher Education*, 2025. Available: <https://doi.org/10.1007/s12528-025-09461-5>. [Accessed: 09-Mar-2026].
- [2] Safe Exam Browser, *Safe Exam Browser – About*. Available: https://safeexambrowser.org/about_overview_en.html. [Accessed: 09-Mar-2026].
- [3] JetBrains, *Create a WebSocket application in Kotlin with Ktor*. Available: <https://ktor.io/docs/server-create-websocket-application.html>. [Accessed: 09-Mar-2026].
- [4] SQLite, *Atomic Commit in SQLite*. Available: <https://sqlite.org/atomiccommit.html>. [Accessed: 09-Mar-2026].
- [5] Microsoft, *Platform Invoke (P/Invoke) - .NET*. Available: <https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke>. [Accessed: 09-Mar-2026].